

Soluciones razonadas del cuaderno

E01-E04

E01. Path y query seleccionan operación/tamaño; Host selecciona vhost; cookie transporta sesión; JSON se deserializa y url puede llegar a un cliente HTTP. crop debe terminar en validación numérica. El error habitual es tratar todo como "parámetros HTTP" sin separar parsers.

E02. Source: request.args. Transformación: replace. El texto final llega a una shell por shell=True; subprocess.run es el sink. Quitar un separador no garantiza que el valor sea un único host ni elimina la gramática de shell. La alternativa es evitar shell, usar argv y validar el tipo de host.

E03. Hay un indicio de error dependiente de comilla, no SQLi confirmada. Compara entradas que ejerciten sintaxis válida/inválida, revisa mensaje/log del laboratorio y usa control con carácter especial no SQL. Mitigación: parametrización si el sink es SQL.

E04. HTTP, proxy o servicio propio son hipótesis. Una petición HTTP, una negociación del protocolo esperado y fingerprinting de banner/comportamiento las separan. "8080 = HTTP" es solo priorización.

E05-E10

E05. En el primer caso la entrada ocupa una expresión numérica; en el segundo está dentro de literal y necesita cerrar/reabrir o neutralizar sufijo. Pares condición verdadera/falsa deben respetar el motor y contexto. Se confirma por diferencia reproducible, no por extracción inmediata.

E06. A: la shell interpreta ; y puede ejecutar un segundo comando. B: todo es el último argumento de ping. Un marcador que produzca error de host literal en B y secuencia en A discrimina. Defensa: B con validación semántica.

E07. Desde /srv/reports/2026/julio/: ../../../../marker.txt. Normaliza julio -> 2026 -> reports -> srv, luego marker. El control inexistente separa lectura de respuesta cacheada. Defensa: ID indirecto y comprobación canónica.

E08. Usa URL del listener propio, registra origen, path y hora; control con puerto cerrado o nombre inexistente. Antes de loopback confirma que el servidor realiza la solicitud, qué esquemas admite y cómo resuelve/redirige. Defensa: política sobre destino efectivo.

E09. La matriz mantiene cuatro dimensiones y cambia una. Aceptación no implica acceso público; acceso no implica ejecución. Mitigación: nombre generado, fuera de webroot, bytes inspeccionados y handler sin ejecución.

E10. Está confirmada evaluación de expresiones compatible con una familia de sintaxis, no RCE ni motor exacto. Identifica motor por errores, documentación y comportamiento seguro; luego evalúa sandbox y exposición.

E11-E16

E11. Validación ocurre antes de URL decode, por eso compara una forma distinta de la consumida. La defensa decodifica una vez de forma definida, canonicaliza y comprueba confinamiento. Verifica con formas equivalentes que todas terminan rechazadas.

E12. No hay confirmación: la latencia base tiene un outlier similar. Alterna múltiples controles y pruebas, usa demora bastante mayor y analiza distribución/mediana. La mitigación depende del sink, no del canal temporal.

E13. Debe conocerse quién posee cada factura y conservar identidad/sesión. Confirma cuando B obtiene un objeto de A que debería estar prohibido y un control propio funciona. Cambiar IDs sin modelo de propiedad puede producir falso positivo.

E14. Confirma la regla exacta y ejecuta una operación inocua documentada que revele identidad, evitando impacto persistente. La capacidad procede de las funciones de find; otro binario tiene otra gramática. Mitigación: regla mínima sin escapes y binario específico.

E15. No basta. Debe existir un segmento candidato escribible, capacidad de colocar el ejecutable y oportunidad de reinicio/arranque. Comprueba ACL de cada ruta y control del servicio. Mitigación: comillas, ACL estrictas y cuenta mínima.

E16. El error indica desfase de reloj. Compara hora con DC y sincroniza dentro del laboratorio; después repite. Un error de permisos posterior es autorización, mientras que KRB_AP_ERR_SKEW impide validar autenticación.

E17-E20

E17. Secuencia: baseline autenticada, mismo objeto con segundo usuario, objeto propio del segundo, método alternativo, campo oculto modificado y sesión invalidada. Cada paso conserva todo lo demás. La evidencia es acceso/acción no autorizada, no una diferencia estética.

E18. Ejecuta identidad/sistema y después cada categoría. Avanza solo con regla sudo, binario SUID/capability relevante, cron con componente controlable, grupo peligroso o secreto reutilizable. La ausencia documentada descarta temporalmente, no el sistema completo.

E19. Primero resuelve el nombre mediante hosts/DNS, fingerprint HTTP y vhosts; en paralelo confirma SMB anónimo y dialecto, luego contenido web, backups, auth y reutilización solo con credenciales. SSH queda para credenciales válidas. Cada rama se cierra tras controles negativos y falta de nueva evidencia.

E20. Lectura LFI es confirmación. La contraseña en config es secreto; login SSH confirma reutilización. No tener sudo descarta solo ese vector. Cron es indicio hasta comprobar que root lo ejecuta y que un componente cargado, ruta, PATH o archivo es modificable. Un directorio escribible no vuelve escribible el script existente, aunque puede permitir reemplazo si permisos y operación lo permiten. Defensa: secretos separados, permisos mínimos y tareas con rutas/archivos protegidos.

E21-E27: pistas graduadas y soluciones

E21

Pista 1 — Dirección conceptual. Correlaciona quién origina cada petición antes de estudiar destinos especiales.

Pista 2 — Parser/componente relevante. Navegador, backend, caché y validador pueden consumir la URL en momentos distintos.

Pista 3 — Primitiva que debe investigarse. Callback HTTP con token irrepitible y un control que no pueda resolverse.

Solución. Observación: vista previa y respuesta no atribuyen el origen. Sé realmente que algún componente usa la URL. Hipótesis: petición solo cliente; backend; caché; validación sin conexión. Experimento: desactivar vista previa y enviar listener/T1, repetir T1, luego listener/T2 con puerto cerrado equivalente. Si es cliente, solo aparece al renderizar; si es backend, T1 llega aun sin navegador; si hay caché, la repetición no llega; si solo valida, ningún token llega. Resultado simulado: T1 llega desde IP del servidor, repetición no, T2 nuevo sí. Conclusión: fetch server-side con caché por URL. Siguiendo paso: documentar resolución, redirects y política sobre destino efectivo con hosts del laboratorio.

E22

Pista 1 — Dirección conceptual. Escribe la consulta resultante completa antes de tocar el valor.

Pista 2 — Parser/componente relevante. El JSON transporta una expresión SQL que termina antes de AND active=1.

Pista 3 — Primitiva que debe investigarse. Oráculo booleano por pares que difieren en una sola condición.

Solución. Observación: se controla una expresión, no necesariamente un literal aislado. Sé realmente la consulta probable dada por el ejercicio. Hipótesis: expresión aceptada; filtro semántico; parametrización de una gramática propia. Experimento: baseline name = 'paper', verdadero name = 'paper' AND 1=1 y falso name = 'paper' AND 1=2, más name = 'missing'. La sintaxis original conserva ... WHERE name='paper' AND 1=1 AND active=1; JSON solo escapa el transporte; SQL es el parser final; el sink es SELECT; la primitiva es cambiar verdad sin romper gramática. Esperado: verdadero coincide con baseline y falso con cero filas. Resultado simulado: así ocurre de forma repetible. Conclusión: entrada evaluada como expresión SQL; aún no se conoce el motor. Siguiendo paso: mitigar con criterios parametrizados/allowlist, no extraer versión como primer movimiento.

E23

Pista 1 — Dirección conceptual. La literalidad del token es evidencia sobre el parser, no sobre un carácter concreto.

Pista 2 — Parser/componente relevante. Distingue validación de host, shell con quoting y argv directo.

Pista 3 — Primitiva que debe investigarse. Comparar el error, la traza de argumentos y un metacarácter dentro/fuera de comillas.

Solución. Observación: `; printf TOKEN` permanece dentro del nombre rechazado y no hay demora. Sé realmente que no se observó una segunda orden. Hipótesis H1: `execve`/lista argv; H2: shell pero valor entre comillas; H3: validador rechaza antes del sink; H4: shell ejecuta pero el canal temporal está aislado. Experimentos: registrar argv en clon local autorizado; comparar host válido con espacio y con comillas; usar un valor deliberadamente inválido sin metacarácter; comparar demoras alternadas con controles. H1 predice un único argumento literal; H2, una orden válida pero dato protegido; H3, ausencia total del proceso; H4, proceso y efecto fuera del canal. Resultado simulado: traza `argv[3]="127.0.0.1; printf TOKEN"`. Conclusión: shell ausente; cambiar `; por &&` no crea gramática. Siguiendo paso: estudiar si el programa interpreta opciones u operandos.

E24

Pista 1 — Dirección conceptual. Recorre el ciclo almacenar → recuperar → interpretar en vez de llamar "shell" al archivo.

Pista 2 — Parser/componente relevante. Multipart, nombre almacenado, cabeceras de descarga, navegador y autorización son capas distintas.

Pista 3 — Primitiva que debe investigarse. Marcador inerte con hash, dos identidades y dos contextos de recuperación.

Solución. Observación: el servidor acepta y devuelve bytes como adjunto; otra cuenta recibe 403. Sé realmente que existe almacenamiento autenticado y recuperación fiel. Hipótesis: subida arbitraria limitada; lectura pública; renderizado same-origin; handler server-side; autorización correcta. Experimento: subir marcador SVG sin script, comparar nombre/hash, solicitar como propietario/anónimo/otra cuenta, inspeccionar Content-Type, Content-Disposition y si se embebe o descarga. Cada hipótesis tiene señal independiente: 201 confirma aceptación; hash confirma bytes; matriz de identidades confirma autorización; cabeceras y navegación confirman tratamiento; solo una ruta con intérprete demostraría ejecución. Resultado simulado: UUID, adjunto, nosniff, 401/403 y ningún render. Conclusión: almacenamiento de archivo, no ejecución. Siguiendo paso: revisar procesadores asíncronos y mantenerlo fuera de webroot.

E25

Pista 1 — Dirección conceptual. Un espacio solo crea otro argumento si alguna capa realiza splitting.

Pista 2 — Parser/componente relevante. Primero reconstruye argv; después estudia el parser de opciones de fetch.

Pista 3 — Primitiva que debe investigarse. Segundo operando u opción, sin necesidad de shell.

Solución. Observación: dos URLs producen dos callbacks y los metacaracteres no actúan. Sé realmente que hay varios operandos y ninguna evidencia de shell. Hipótesis: `split simple`; `shlex.split`; parser propio; shell con quoting. Experimento: `"https://a/x y"`, `https://a/x\ https://b/y`, token con espacios y `--help/--` en clon inocuo. `shlex.split` preserva comillas y escapes; `split simple` no; lista de un elemento no produce dos callbacks; el parser de opciones decide si `-x` es opción, no el shell. Resultado simulado: comillas y barra preservan un elemento y `--version` cambia la salida. Conclusión: `shlex.split` seguido de option parsing: inyección de argumentos, no de comandos. Siguiendo paso: construir argv fijo, validar cada URL e insertar `--` si la herramienta lo soporta y los operandos quedan limitados.

E26

Pista 1 — Dirección conceptual. La misma cadena se valida al guardar y se consume horas después.

Pista 2 — Parser/componente relevante. Parser URL, resolvedor DNS, redirect handler y cliente indicado por User-Agent.

Pista 3 — Primitiva que debe investigarse. Petición saliente atribuible y cambio del destino efectivo entre comprobación y conexión.

Solución. Observación: proceso nocturno hace callback, sigue al menos un redirect y presenta cliente curl; DNS alternante cambia el resultado. Sé realmente que hay una petición server-side diferida y que la herramienta ofrece

redirects en esa configuración. Hipótesis: se valida solo al guardar; se resuelve en ambos momentos; caché/pinning; cada salto se revalida; política textual. Experimentos: tokens por fase, host estable, redirect entre dos hosts del laboratorio, TTL controlado y logs de IP efectiva. Cada hipótesis predice distinta consulta DNS y callback. Resultado simulado: allowlist al guardar, nueva resolución al ejecutar y redirect sin revalidación. Conclusión: la política no se aplica al destino efectivo. Siguiente paso: permitir esquemas explícitos, resolver y validar todas las direcciones, conectar de forma vinculada y revalidar cada redirect; estudiar capacidades de la versión/configuración observada de curl sin asumir protocolos.

E27

Pista 1 — Dirección conceptual. Prioriza la relación que combina consumidor root y componente escribible.

Pista 2 — Parser/componente relevante. La lista argv, sudoers, el shell del script y el PATH del timer son cuatro parsers/controles distintos.

Pista 3 — Primitiva que debe investigarse. Resolución de un nombre sin ruta dentro del PATH efectivo de root, demostrada con marcador inocuo.

Solución. Observación: el renderizador muestra argv fijo; sudo autoriza una invocación exacta como archive; root ejecuta un script protegido que busca rotate-report; el primer directorio de PATH es escribible por el grupo del alumno. Sé realmente que existe una ruta candidata, no que el timer esté activo ni que use ese directorio. Hipótesis: H1 PATH hijacking en timer; H2 PATH saneado o comando resuelto por ruta/caché; H3 sudo permite argumentos extra; H4 renderizador interpreta opciones del archivo. Experimentos máximos: (1) id, grupos y ACL; (2) sudo -l y control con argumento distinto; (3) observar dos ciclos/UID y entorno del timer; (4) inspeccionar llamada exacta; (5) comprobar orden y escritura de /opt/cleanup/bin; (6) colocar rotate-report de marcador que registre id; (7) esperar ciclo y comparar control sin marcador; (8) retirar el archivo y confirmar restauración. H1 predice /tmp/rotate-proof UID 0; H2, ningún marcador y ejecución del legítimo; H3 se descarta si sudo rechaza otro argumento; H4 se descarta porque argv muestra paths generados, aunque aún requeriría estudiar opciones de la herramienta. Resultado simulado: marcador UID 0 solo durante presencia del candidato. Conclusión: capacidad mínima de ejecución mediante PATH heredado del timer; ni script legible ni sudo exacto eran la causa. Siguiente paso: documentar, retirar marcador y mitigar con ruta absoluta, directorios no escribibles y entorno explícito.

Tipo test

1. B: dos condiciones reproducibles forman un oráculo.
2. C: el fragmento normalmente lo retiene el cliente.
3. B: sin shell es parte del argumento.
4. B: la arista exige precondiciones y validación.
5. C: controla la semántica final de la ruta.